Building Guardrails for AI-Generated Frontends: A Platform Engineering Approach

DESIGN SYSTEMS | STORYBOOK | PLAYWRIGHT | AXE-CORE | CI GUARDRAILS

Note: The views expressed in this presentation are my own and do not represent my employer.

Hi, I am Niharika 👋

Frontend architecture • accessibility • AI-assisted development



AI changes the speed, not the standards

The output appears faster than our review systems were designed for.

What AI optimizes for

1. Generate quickly
2. Satisfy the prompt
3. Produce plausible code

What teams still need

- ✓ Accessible behavior
- ✓ Approved components
- ✓ Reliable tests
- ✓ Maintainable code

The gap is where platform engineering can help.

The platform engineering lens

Treat standards as capabilities of the developer platform — not reminders in a wiki.



Paved path = defaults + reusable building blocks + automated feedback + enforceable gates

Three ways to use AI for frontend generation

Same task. Different amount of engineering knowledge encoded into the environment.

1 Generate as-is

Ask for the feature. Accept the generated structure. Review later.

Fastest start
Highest review burden
Standards are implicit

2 Prompt the standards

Add accessibility, testing, and component requirements to the prompt.

Better intent
Still probabilistic
Prompt quality matters

3 Encode guardrails

Constrain generation with components, test harnesses, Storybook contracts, and CI.

Reusable
Repeatable
Closer to a paved path

Approach 1: generate first, review later

The code can be plausible while the engineering debt remains hidden.

Component drift

A custom button or dialog instead of the approved implementation.

Behavior gaps

Focus, error announcements, keyboard flows, and state transitions are easy to miss.

Test gaps

The happy path exists, but meaningful interaction checks do not.

Maintenance cost

Every generated variation creates another pattern the team must understand.

Approach 2: prompting helps — but prompts are not policy

Instructions improve the odds. They do not guarantee the outcome.

Example prompt additions

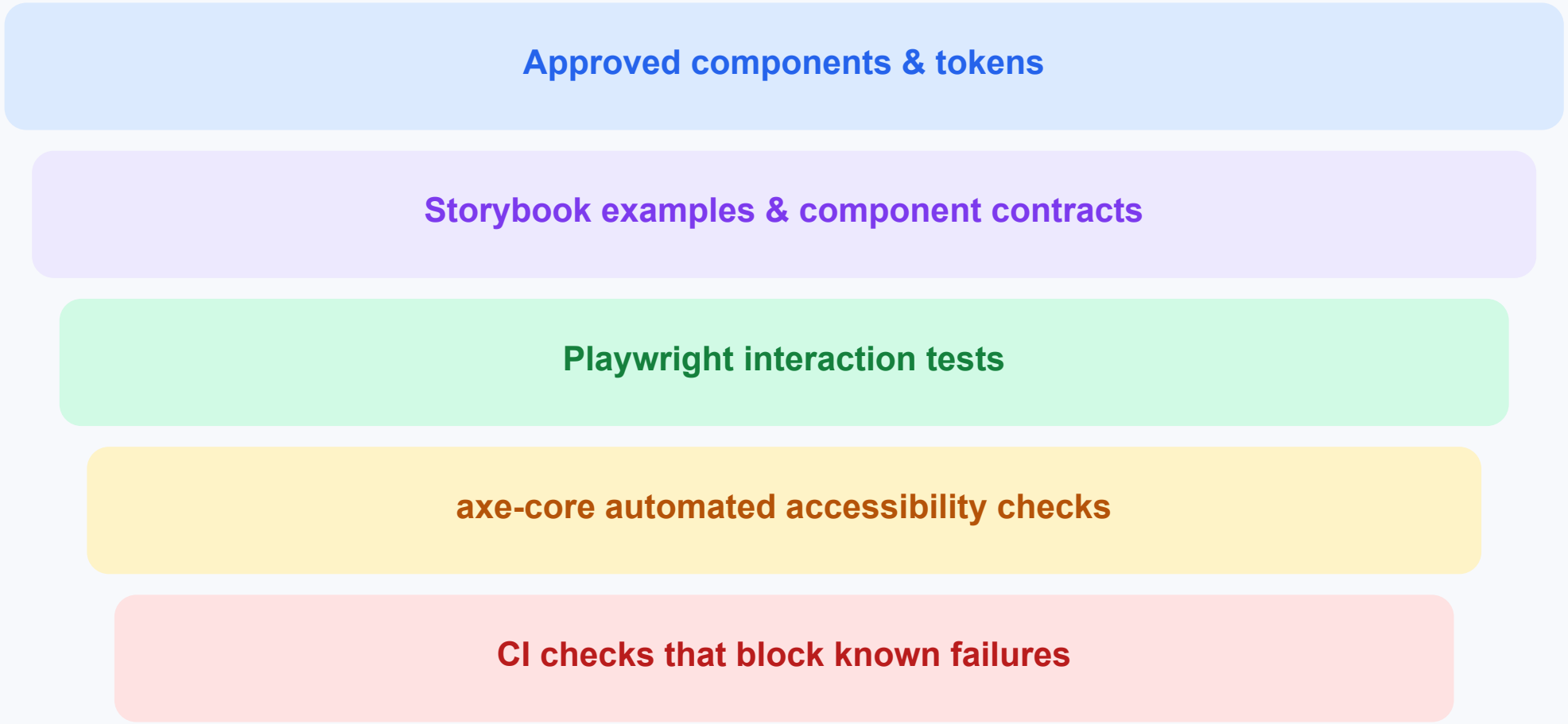
- Use semantic HTML
- Use approved design-system components
- Ensure keyboard access
- Add accessible names and errors
- Write Playwright tests

Why it still varies

Context	Does the model know your components?
Specificity	Did the prompt name every important rule?
Consistency	Will the same prompt produce the same quality tomorrow?
Verification	Who proves the requirements were actually met?

Approach 3: architecture-level guardrails

Move important expectations from “please remember” to “this is how the platform works.”



The AI can still generate code — but inside a narrower, safer solution space.

A guardrail stack for AI-generated frontends

Each layer catches a different class of failure.



Feedback should arrive as close as possible to generation — not at the end of the release cycle.

Design systems become executable policy

The most valuable guardrail is often the component you do not have to re-implement.

Without a constrained component

AI may invent:

- markup
- focus logic
- keyboard behavior
- error treatment
- styling variants

With a platform component

AI chooses the component and configures intent:

`<ds-dialog>`	focus + ESC + return focus
`<ds-field>`	label + error relationships
`<ds-button>`	states + semantics + styling

Storybook makes hidden states visible

Generated code needs examples of the states that matter — not only the default render.

Default

baseline

Error

validation

Keyboard

focus

Loading

async

Empty

edge case

Component stories can become context for humans, tests, and eventually coding agents.

Playwright + axe-core: verify behavior, not just markup

Automated checks are strongest when they exercise real user flows.

Playwright can verify

- ✓ Tab order and keyboard flow
- ✓ Dialog open / close / focus return
- ✓ Validation and error states
- ✓ Critical navigation paths

axe-core can catch

- ✓ Missing accessible names
- ✓ Invalid ARIA relationships
- ✓ Certain contrast / structure issues
- ✓ Common rule violations at scale

Neither replaces manual accessibility testing.

CI turns guidance into a delivery contract

A guardrail becomes meaningful when failure changes what happens next.



If the same class of failure escapes repeatedly, promote the lesson into a reusable rule or test.

What changes across the three approaches?

A qualitative comparison — not a promise that guardrails eliminate defects.

Dimension	Generate as-is	Prompt standards	Architecture guardrails
Speed to first code	High	High	High
Consistency	Low	Medium	High
Dependence on prompt quality	Low	High	Lower
Automated feedback	Low	Medium	High
Review burden	High	Medium	Lower
Up-front platform investment	Low	Low	Higher

Where guardrails still fall short

Some problems require context, judgment, and real users.

Semantics in context

A technically valid control can still be confusing in the task flow.

Screen reader experience

Announcements, reading order, and interaction quality need human evaluation.

Visual / cognitive usability

Automation cannot decide whether an interface is understandable.

Novel failures

Guardrails only catch what the platform knows how to express.

Guardrails reduce avoidable variation. Engineering judgment remains part of the system.

A practical adoption path

Start with the failures you already see. Encode one layer at a time.

1. **Identify recurring review findings** — What keeps appearing in PRs, audits, or QA?
2. **Move stable rules into components** — Make the correct implementation the easiest option.
3. **Add reusable behavior tests** — Test keyboard flows, errors, focus, and critical paths.
4. **Gate known failures in CI** — Create fast feedback before manual review.
5. **Feed escaped bugs back into the platform** — Bug → rule → test → prevention.

The goal is not to make AI write perfect code.

The goal is to build a platform where better code is the path of least resistance.

Thank you

Niharika P. Pujari • linkedin.com/in/niharika-p-pujari